

Reviewer Pack — Minimal Topological Entropy of Decision Trees and SAT Clause...

Entry 9edfb2f7ac10 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 23:23:00 UTC

Minimal Topological Entropy of Decision Trees and SAT Clause Subset Complexity

Entry ID: 9edfb2f7ac10 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 23:23:00 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics (Entropy Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

For any given satisfiable Boolean formula φ with n clauses, the minimal topological entropy ($h_{\min}(\varphi)$) of its decision tree is linearly proportional to its SAT clause subset complexity ($c_{\text{sub}}(\varphi)$), such that $h_{\min}(\varphi) = \Theta(c_{\text{sub}}(\varphi))$.

Rationale (proposer's reasoning):

The minimal topological entropy provides a measure of the information content or complexity of dynamical systems. In the context of Boolean satisfiability, it may capture the complexity of the decision-making process in the tree structure representing the search space for solutions. By linking this to SAT clause subset complexity, we may expose a deeper connection between the structural complexity of the formula and the efficiency of its resolution.

Taxonomy category: topological_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c2c9f563046e68c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of the generated Boolean formulas ($n \geq 50$) show a correlation coefficient $r \geq 0.9$ between minimal topological entropy and clause subset complexity.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - topological dynamics AND entropy theory AND Boolean satisfiability - minimal topological entropy AND decision tree AND SAT clause subset complexity - linear proportionality AND $h_{\min}(\varphi)$ AND $c_{\text{sub}}(\varphi)$ IN BOOLEAN Satisfiability

Top relevant hits considered: - [http://arxiv.org/abs/2110.01541v4] Dynamical entropy of probability measures on infinite product spaces - [http://arxiv.org/abs/1612.05917v1] Linear Quantum Entropy and Non-Hermitian Hamiltonians - [http://arxiv.org/abs/1801.02665v3] Symbolic relative entropy in quantifying nonlinear dynamics of equalities-involved heartbeats - [http://arxiv.org/abs/quant-ph/0008095v3] Entropy lower bounds of quantum decision tree complexity - [http://arxiv.org/abs/2403.07054v3] Minimal Fractional Topological Insulator in half-filled conjugate moiré Chern bands - [http://arxiv.org/abs/1312.3003v1] Decision Trees, Protocols, and the Fourier Entropy-Influence Conjecture - [http://arxiv.org/abs/1003.1544v2] Linear Mappings of Free Algebra - [http://arxiv.org/abs/1006.1021v1] A Gruss inequality for n-positive linear maps

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_formula(n):
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
        clauses.append(clause)
    return clauses

def construct_decision_tree(clauses):
    if not clauses:
        return "leaf"

    counts = {var: 0 for var in set(abs(var) for clause in clauses for var in clause)}
    for clause in clauses:
        for var in clause:
            counts[abs(var)] += 1

    decision_var = max(counts, key=counts.get)
    left_clauses = [clause for clause in clauses if decision_var in clause]
    right_clauses = [clause for clause in clauses if -decision_var not in clause]
```

```

    return {
        "var": decision_var,
        "left": construct_decision_tree(left_clauses),
        "right": construct_decision_tree(right_clauses)
    }

def calculate_topological_entropy(tree):
    if tree == "leaf":
        return 0

    left = tree["left"]
    right = tree["right"]

    p_left = Fraction(1, 2) * (len(left) / len(clauses))
    p_right = Fraction(1, 2) * (len(right) / len(clauses))

    entropy = -p_left * math.log2(p_left) - p_right * math.log2(p_right)
    return entropy

def calculate_clause_subset_complexity(clauses):
    n = len(clauses)
    complexity = sum(1 << i for i in range(n + 1)) - 1
    return complexity

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_max = 40
    instances_tested = 0
    metric_values = []

    for n in [5, 10, 15, 20, 30, 40]:
        if instances_tested >= 30:
            break

        for _ in range(5):
            clauses = generate_formula(n)
            tree = construct_decision_tree(clauses)
            entropy = calculate_topological_entropy(tree)
            complexity = calculate_clause_subset_complexity(clauses)

            metric_values.append((entropy, complexity))
            instances_tested += 1

    if instances_tested < 30:
        return {
            "metric_name": "Topological Entropy vs Clause Subset Complexity",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    mean_entropy = sum(val[0] for val in metric_values) / len(metric_values)
    mean_complexity = sum(val[1] for val in metric_values) / len(metric_values)

```

```

correlation_coefficient = sum((val[0] - mean_entropy) * (val[1] - mean_complexity) for val in
return {
    "metric_name": "Topological Entropy vs Clause Subset Complexity",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.9,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None) /
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results if res["m
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]),
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient_too_low' first_failing_s

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 111, in <module>
    result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 75, in run_trial
    tree = construct_decision_tree(clauses)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 40, in construct_dec
    "left": construct_decision_tree(left_clauses),
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 40, in construct_dec
    "left": construct_decision_tree(left_clauses),
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 40, in construct_dec
    "left": construct_decision_tree(left_clauses),

```

```

[Previous line repeated 994 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ef8f8e3.py", line 29, in construct_dec
    counts = {var: 0 for var in set(abs(var) for clause in clauses for var in clause)}

```

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic reasoning:

9. Final verdict & safety rail

Reasoning:

11. Audit log (LLM calls)

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12228
2	propose	ollama_remote	glm4:latest	0	0	12965
3	preregistration	ollama_remote	glm4:latest	0	0	9012
4	novelty	ollama_remote	glm4:latest	0	0	12514
5	novelty	ollama_remote	glm4:latest	0	0	9484
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17511
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36455
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15395
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13372
10	judge	ollama_remote	glm4:latest	0	0	65583

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 204520 ms total latency. Provider mix: {'ollama remote': 10}

12. Reproducibility

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9edfb2f7ac10.tar.gz (if generated)